KABIR KANHA ARORA
1710110165
CSE

ALGORITHMS ASSIGNMENT #5

A10: a) CLIQUE

Given a graph $G(V,E)$ and an integer $k \leq |V|$, we need to decide whether there is a clique of size $\geq k$

Hence,

We return 1 iff there exists a set of vertices $S \subseteq V$ and $(u,v) \in E \ \forall \ u,v \in S$

We know that to prove a problem to be NP-complete:

(i) Prove it is NP

(ii) Show every known NP problem reduces to it in polynomial time.

PROOF

I. Given a graph $G(V,E)$, k and S, we check whether $|S| \geq k$ and for all pairs $u,v \in S$, if $(u,v) \in E$. This can be done in $O(n^2)$. Since this verification can be done in polynomial time, it is clearly NP

classmate

Scanned with CamScanner

II. The second part is harder to prove. Instead of showing that every NP problem reduces to it in polynomial time, clearly we can pick a known NP-complete problem and show that it can be reduced to the clique problem.

Let us try reducing the 3-SAT problem, which is known to be NP-complete to CLIQUE.
SAT stands for the boolean ~~SAT~~ satisfiability problem and 3 refers to the number of literals per clause in CNF.

Let τ be a 3-SAT instance. $C_1, C_2, \ldots C_m$ are the clauses of τ defined over the variables $\{x_1, x_2, \ldots x_n\}$. From τ, we will produce a graph $G(V,E)$ and an integer k s.t. G has a clique of size k iff τ is satisfiable.
We represent every clause C_i as $\{z_{i_1}, z_{i_2}, \ldots, z_{i_t}\}$ where each z_{ij} represents a literal at since it is 3-SAT, $t \leq 3$.
We construct a graph $G(V,E)$ by adding t vertices for every clause $C_i = \{z_{i_1}, z_{i_2}, \ldots z_{i_t}\}$
Time taken $\rightarrow O(tm) \in O(m)$ $(t \leq 3)$

classmate

Scanned with CamScanner

For every pair of vertices v_{ab}, v_{cd} in G,
we add the edge (v_{ab}, v_{cd}) iff:

$a \neq c$

$z_{ab} \neq \overline{z_{cd}}$

Implies both literals can be satisfied simultaneously.

Implies that the literals belong to different clauses

The final step is to determine k.
We set $k = m$, the no. of clauses in τ.

Claim: τ is satisfiable iff G as constructed above has a clique of size $k \geq m$.

First,
Suppose τ is satisfiable
$\Rightarrow \exists$ an assignment $(x_1^*, x_2^*, \ldots, x_n^*)$ s.t.
every clause C_i in τ is satisfied.
Note that it is enough to satisfy any one of the literals from every clause to satisfy the clause itself.
We select one literal per clause to satisfy which we denote as $(z_1^*, z_2^*, \ldots z_m^*)$.
Let $v_1, v_2, \ldots v_m$ be the corresponding vertices.

classmate

Scanned with CamScanner

Notice that all $z_1^*, z_2^* \ldots z_m^*$ have the same truth assignment, since otherwise $\exists\, i,j\ z_i^* = \overline{z_j^*}$
The set $S = \{v_1, v_2, \ldots v_m\}$ must now form an m-clique.

Conversely,

Suppose G has a clique of size $m \geq k$.
Let $v_1, v_2 \ldots v_q$ be a clique of size $q \geq m$
The first m edges also form a clique.
Since there are no connecting edges between vertices of the same clause, every v_i corresponds to a literal z_i from exactly one clause C_i
Since $v_1 \ldots v_m$ is a clique, the corresponding literals z_i, z_j of any pair $v_i, v_j \in \{v_1, \ldots v_m\}$ can be satisfied immediately, simultaneously (by construction)
Now to construct and assignment to satisfy τ, we just need to satisfy $z_1 \ldots z_m$ and assign the rest randomly.
Every C_i is satisfied now, and hence τ is satisfied.

$\therefore$ Hence, Proved.

classmate

Scanned with CamScanner

b) SUBGRAPH ISOMORPHISM

Given a one-one onto f^n f from $V_1 \to V_2$, we can verify if this is indeed a one-one onto f^n and then take each edge $(u,v) \in G_1$ and verify that $(f(u), f(v)) \in G_2$.
Clearly, this can be done in $O(n+m)$

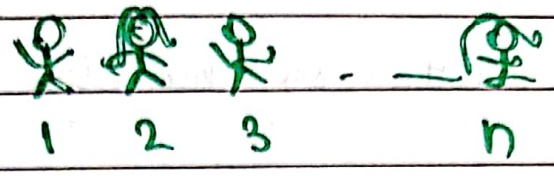
Let us now prove CLIQUE $\leq_p$ subgraph-isomorphism.
Let $\langle G, k \rangle$ be the input to the clique. For the subgraph isomorphism, we let G_1 be a complete graph on k vertices and let $G_2 = G$.
This can be done in polynomial time
By def^n of a clique being a complete subgraph on k vertices, there is a clique of size k in G iff G_1 is a subgraph of G_2, i.e; if there is a clique of k vertices in G, then mapping the vertices of G_1 to those in G_2 gives a sol^n for the subgraph isomorphism problem.
Similarly, if there is a sol^n of the subgraph isomorphism problem, then the vertices in G_2 mapped to by the vertices in G_1 form a clique of k-vertices

Hence, Proved.

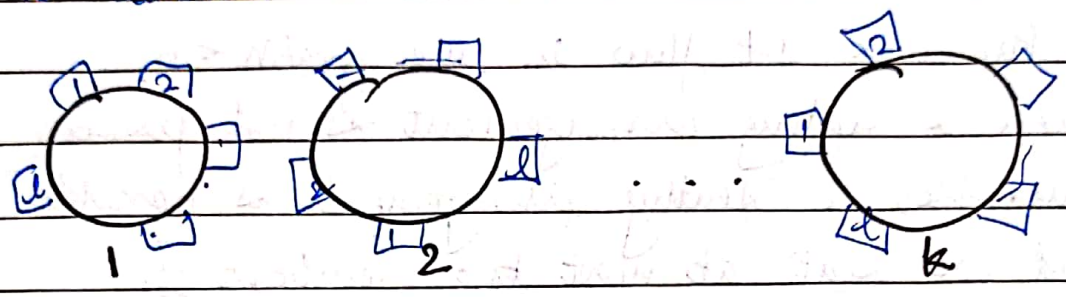
classmate

Scanned with CamScanner

A1: • n friends



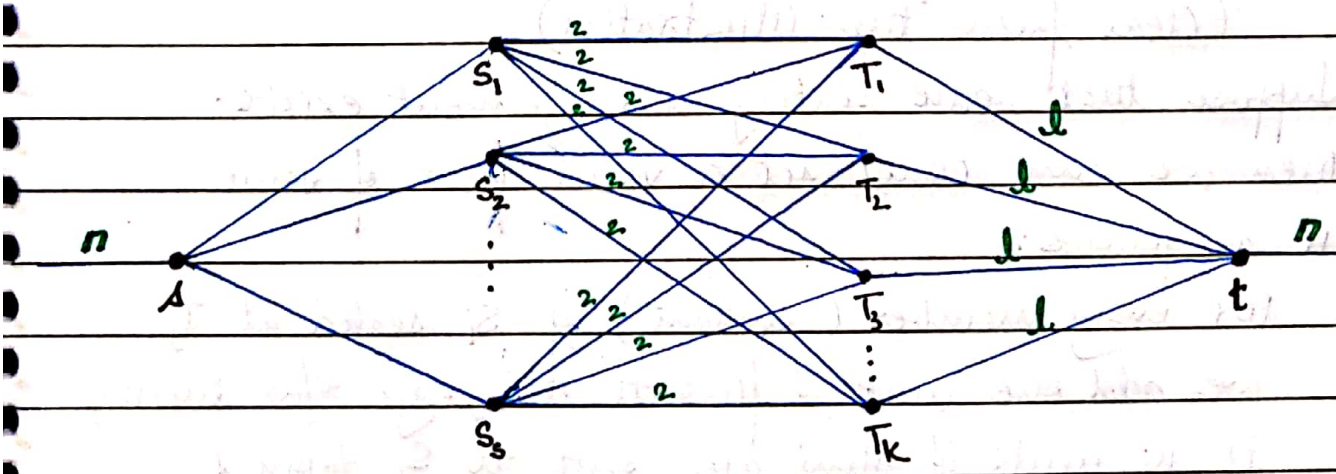
• k tables



• s schools

Rule: AT MOST two people from one school on one table.

We see that, for this problem, a graph G can be constructed as follows:



where,

$S_1, S_2 \ldots S_s$ represent the schools

$T_1, T_2 \ldots T_k$ represent tables

s and t represent the source and sink respectively.

classmate

Scanned with CamScanner

The numbers written on the edges denote the capacities

* If the max s-t flow in the graph < n, then a seating arrangement is not possible.
* Otherwise, a seating arrangement is possible and we seat at most two members of school S_i at table T_j, depending upon the value of flow on their edge.

NOTE: All values of flows on edges must be integral.

Proof of correctness

Firstly, max flow is never > n (Clear from the illustration).

Suppose that some seating arrangement exists. Then we can construct a valid flow of value n as follows:

For every member (student) of S_i seated at T_j, we add one unit of flow to the edge b/w them.

If n_i units of flow are sent to S_i from s, the conservation is satisfied at each S_i.

For each T_j, we add as many units of flow to t as there are people at the table (at most l)

All this means that if max flow < n, no seating arrangement exists.

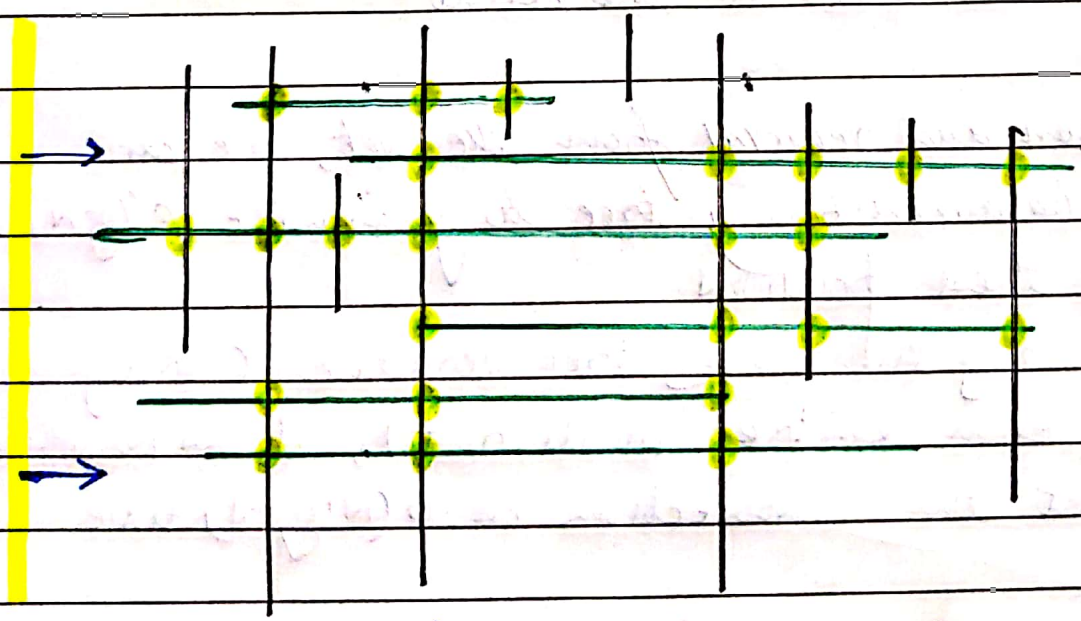
classmate

Scanned with CamScanner

Now, suppose some flow value A exists.
By the properties of Ford-Fulkerson algorithm, there is an integral flow of value A. We can construct a seating arrangement as described above.
The edges out of s add up to n. Conservation of flow at each S_i indicates that each student is seated. Conservation of flow at each T_j indicates that each table seats at most l people.
$\Rightarrow$ Valid seating arrangement.

Time to construct the graph $= O(|V| + |E|) = O(sk)$
Running Ford Fulkerson n times $\rightarrow O(n(|V| + |E|)) = O(nsk)$

A6:



Assumption: Coincident end-points do <u>not</u> count as an intersection.

classmate

Scanned with CamScanner

Algo:

1. Sweep from L → R. When the start of any horizontal line is encountered, we add the starting coordinates to a set we maintain.
2. When an end pt. of a horizonal line is encountered, we remove its starting point from the set.

Note: We assume that there are no overlapping lines, although this has no impact on the NUMBER of intersections.

3. When we hit a vertical line, denoted by (x_1, y_1) and (x_1, y_2), we check for all horizontal segments in our set in the range (y_1, y_2) and add them to the count.

For addition and removal from the set, we can use a balanced binary tree to guarantee $O(\log N)$ time for these operations

By augmenting the binary tree structure (storing size of each subtree in its root), it is possible to count the intersections in $O(N \log N)$ time.

classmate

Scanned with CamScanner

A3: <u>No</u>

The KMP algorithm will <u>not</u> work correctly.
An example is sufficient to show the same.

String: ABX AABXBXAB
Pattern: AB* AB
f: 0 0 1 1 2

→ The first mismatch happens at ABX AABXBXAB
Using f, we go to index 1 in the pattern and start matching. Another mismatch sends us to index 0, where we have a match. The next mismatch occurs at ABX AABXBXAB, and we go to index 1 in the pattern and the rest of the pattern is succesfully matched.
So we report that a pattern has been found in the last 5 characters of the string, which is clearly INCORRECT.

→ A workaround for this could be to set a flag for every * that consumes a character. This flag is active for n chars in the pattern, where n is the no. of chars before it. In case a mismatch happens in the next n chars, we get back to the symbol consumed by * and start checking from it again from the beginning of the pattern.

classmate

Scanned with CamScanner